

ran), then iterate through the list of events. Each event reported to the program is an object, and every event object has a type. If no event has happened, this section is skipped over.

In this minimal program, where the only action a user can take is to close the window, the only event type we check for is the constant `pygame.QUIT`, generated by pygame when the user clicks the close button. If we find this event, we tell pygame to quit, which frees up any resources it was using. Then we quit our program.

8. Do any “per frame” actions.

In this section we’ll eventually put any code that needs to run in every frame. This might involve moving things in the window or checking for collisions between elements. In this minimal program, we have nothing to do here.

9. Clear the window.

On each iteration through the main loop, our program must redraw everything in the window, which means we need to clear it first. The simplest approach is to just fill the window with a color, which we do here with a call to `window.fill()`, specifying a black background. We could also draw a background picture, but we’ll hold off on that for now.

10. Draw all window elements.

Here we’ll place code to draw everything we want to show in our window. In this sample program there is nothing to draw.

In real programs, things are drawn in the order they appear in the code, in layers from backmost to frontmost. For example, assume we want to draw two partially overlapping circles, A and B. If we draw A first, A will appear behind B, and portions of A will be obscured by B. If we draw B first and then A, the opposite happens, and we see A in front of B. This is a natural mapping equivalent to the layers in graphics programs such as Photoshop.

11. Update the window.

This line tells pygame to take all the drawing we’ve included and show it in the window. Pygame actually does all the drawing in steps 8, 9, and